Q1. What is Security?

Answer:

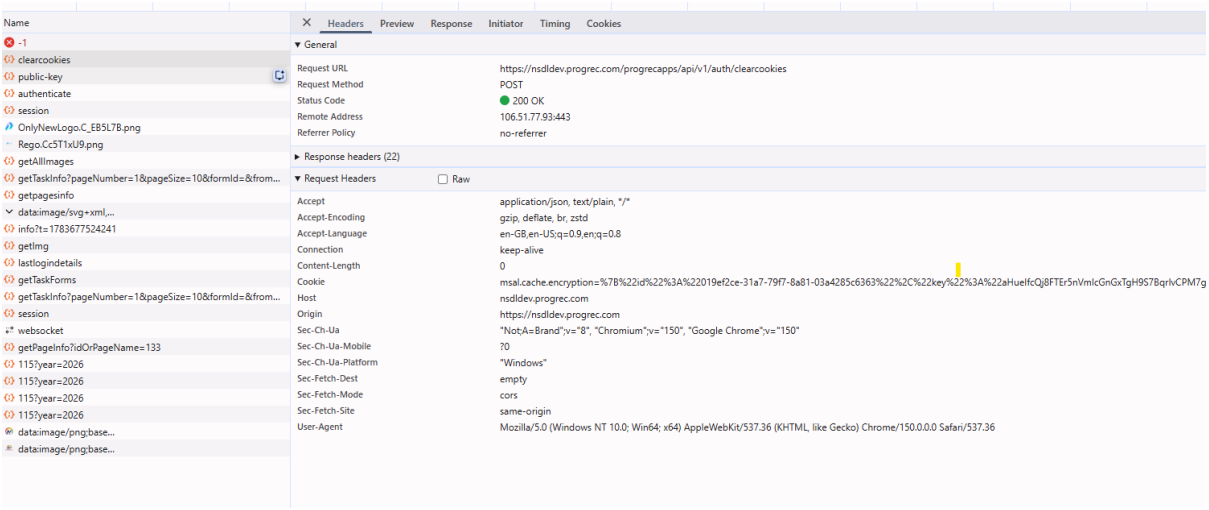
Security is the practice of protecting an application's data, users, and resources from unauthorized access, misuse, modification, disclosure, or destruction. In enterprise applications, security is implemented using authentication, authorization, encryption, secure communication (HTTPS), input validation, and auditing to ensure the confidentiality, integrity, and availability of the system.

Q3. What Are the Main Objectives of Security?

Answer:

The three primary objectives are:

- **Confidentiality** – Prevent unauthorized access to data.
- **Integrity** – Ensure data remains accurate and unaltered.
- **Availability** – Ensure systems and data are accessible to authorized users when needed.



Q3. Difference between Authentication and Authorization?

Answer

Authentication verifies the identity of the user, whereas Authorization determines what resources the authenticated user is permitted to access. Authentication always occurs before Authorization.

Q3.a. Which comes first?

Authentication

↓

Authorization

Always.

Q3.b. Can Authorization happen without Authentication?

No.

Because

Spring must first know

Who you are

before deciding

What you can access.

Q3.c. Can Authentication succeed but Authorization fail?

Yes.

Example

Employee logs in successfully.

Attempts

/admin

Employee has

ROLE_USER

Admin page requires

ROLE_ADMIN

↓

Authentication Success

Authorization Failed

↓

403 Forbidden

Common Interview Scenario

Interviewer:

You logged in successfully but got

403 Forbidden

Why?

Answer:

Authentication was successful, but Authorization failed because the authenticated user lacked the required role or authority to access the requested resource.

HTTP Status Codes

Status Code	Meaning	Scenario
200 OK	Request succeeded	User authenticated and authorized
401 Unauthorized	Authentication required or failed	User is not logged in or provided invalid credentials
403 Forbidden	Authorization failed	User is authenticated but lacks the required permission

How is the password encoded in Spring Security?

Spring Security uses a PasswordEncoder implementation such as BCryptPasswordEncoder to hash passwords before storing them in the database. During registration, the application calls passwordEncoder.encode(rawPassword) and stores only the BCrypt hash. During login, Spring Security verifies the password using passwordEncoder.matches(rawPassword, encodedPassword). BCrypt generates a unique random salt for every password, so the same password produces different hashes each time, making it highly secure.

User Authentication Flow

When a user attempts to log in, the authentication process follows these steps:

Step 1: User Login

The user enters their **username** and **password** on the login page.

Step 2: Clear Existing Cookies

Immediately after the login request, the frontend invokes the following API:

/api/v1/auth/clearcookies

This API clears any existing authentication cookies from previous sessions, ensuring that the new login process starts with a clean session and avoids conflicts with old authentication data.

Step 3: Fetch RSA Public Key

Next, the frontend calls:

/api/v1/auth/public-key

This API returns an **RSA Public Key**. The public key is not human-readable because it is generated using the **RSA encryption algorithm**.

The frontend uses this public key to encrypt the user's password before sending it to the server. This ensures that the password is never transmitted as plain text over the network.

Step 4: Authenticate User

After encrypting the password, the frontend sends the login request to:

/auth/authenticate

The encrypted username and password are sent to the backend for authentication.

Step 5: Process Authentication

Inside the backend, the request is handled by the processAuthentication() method.

The first step is to check whether **Multi-Factor Authentication (MFA)** is enabled for the user.

```
if (isMultiFactorAuthenticationEnabled()) {
   // Perform MFA validation
}
```

Step 6: Decrypt the Password

Since the password received from the frontend is encrypted using the RSA public key, it must be decrypted before authentication.

The backend uses the corresponding **RSA Private Key** along with Java's **Cipher** class to decrypt the password.

Example:

```
Cipher cipher = Cipher.getInstance("RSA");
cipher.init(Cipher.DECRYPT_MODE, privateKey);

String decryptedPassword = new String(
   cipher.doFinal(encryptedPasswordBytes)
);
```

After decryption, the original password is available in plain text **only in server memory** for authentication.

Step 7: Authenticate the User

Once the password is decrypted, Spring Security's AuthenticationManager is invoked to verify the user's credentials.

```
authenticationManager.authenticate(
   new UsernamePasswordAuthenticationToken(
      userName,
      decryptedPassword
   )
```

);

During this process, Spring Security:

- Retrieves the user details from the database.
- Compares the entered password with the stored BCrypt hashed password.
- Verifies whether the account is active, unlocked, and enabled.
- If all validations are successful, the user is authenticated.

Step 8: Invalid Credentials

If the username or password is incorrect, Spring Security throws a BadCredentialsException.

throw new BadCredentialsException("Invalid Credentials");

The frontend receives the response and displays:

Invalid Credentials

Step 9: Account Locking

To prevent brute-force attacks, the application tracks the number of failed login attempts.

If a user enters incorrect credentials multiple times (for example, five consecutive failed attempts), the account is automatically locked.

Once the account is locked:

- The user cannot log in, even with the correct password.
- The system returns an appropriate error message indicating that the account is locked.
- The user must contact the **Organization Administrator** to unlock the account.

This mechanism enhances the application's security by protecting user accounts from unauthorized access attempts.

Complete Authentication Flow

Interview Explanation (2–3 Minutes)

When a user logs in, the frontend first clears any existing authentication cookies by calling the clearcookies API. It then requests an RSA public key from the public-key API. The frontend uses this public key to encrypt the user's password before sending it to the backend, ensuring that the password is never transmitted in plain text.

On the backend, the processAuthentication() method handles the request. It first checks whether Multi-Factor Authentication (MFA) is enabled. Since the password is RSA-encrypted, the backend decrypts it using the corresponding RSA private key and Java's Cipher class. After decryption, the application invokes AuthenticationManager.authenticate() with a UsernamePasswordAuthenticationToken. Spring Security validates the username, compares the decrypted password with the BCrypt hash stored in the database, and checks whether the account is active and unlocked. If authentication succeeds, the user is logged in. If authentication fails, a BadCredentialsException is thrown. Additionally, the application tracks failed login attempts, and after a configured limit, the user's account is locked to protect against brute-force attacks.